

Assignment 2

Nurzhan Zeinulla SE-2506

Factory Method and Abstract Factory

Introduction

Option A uses Factory Method for Truck and Ship. A creator owns the delivery workflow while subclasses choose the transport. Abstract Factory supplies matching Button and Checkbox objects for Windows or Mac.

Clean Code: five principles

1. Meaningful names: RoadTransportFactory identifies a creator; Truck identifies a product.

Example: `class RoadTransportFactory extends TransportFactory`

2. Polymorphism: the delivery workflow has no product-selection if/switch.

Example: `Transport transport = createTransport(); transport.deliver();`

3. Small methods: a factory method has only one construction responsibility.

Example: `public Button createButton() { return new WindowsButton(); }`

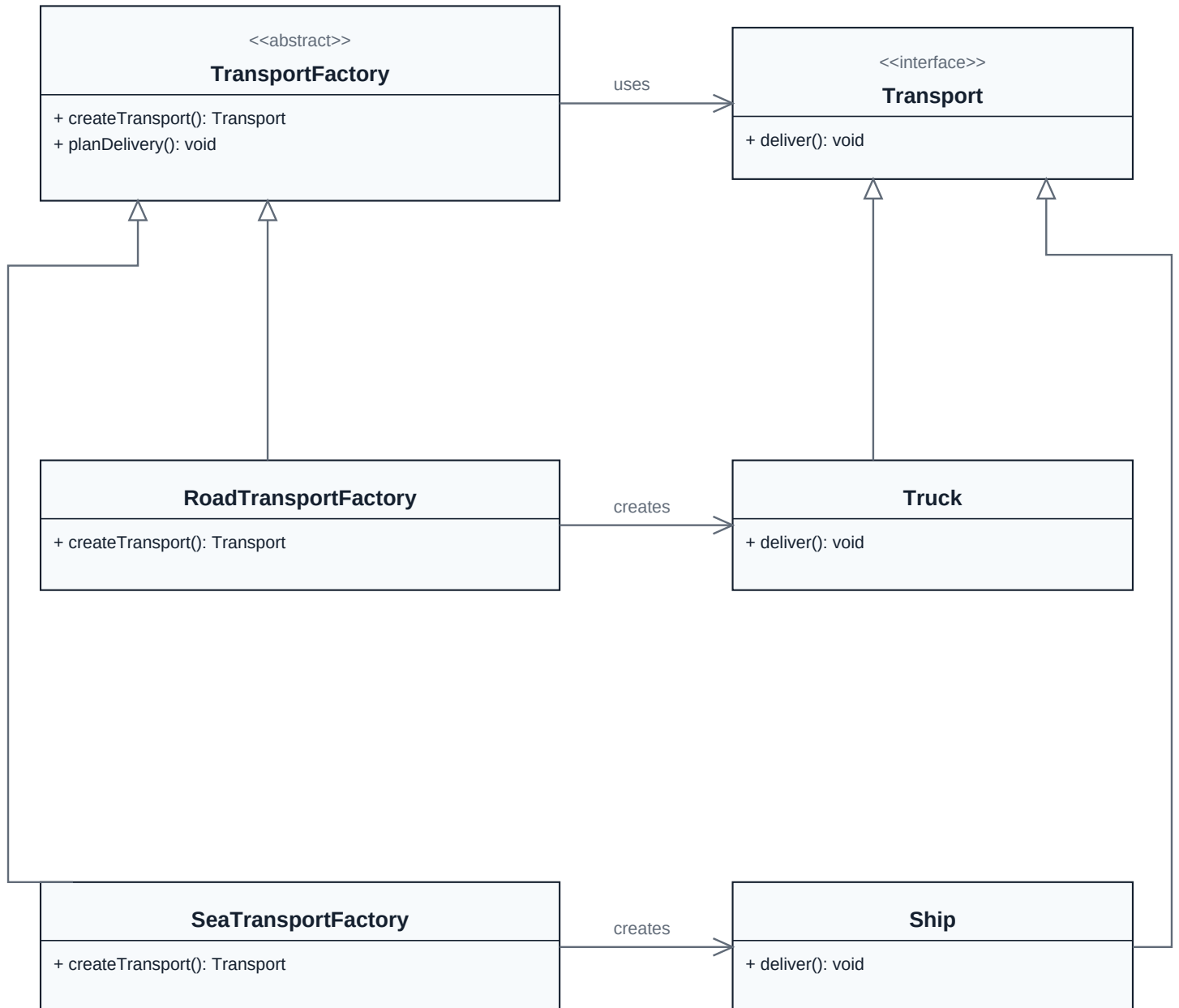
4. No duplicated assembly: Application creates the pair once through GuiFactory; both platforms reuse this client.

Example: `button = factory.createButton(); checkbox = factory.createCheckbox();`

5. Encapsulation: the client keeps its product references private and final. Callers use behavior instead of replacing its fields.

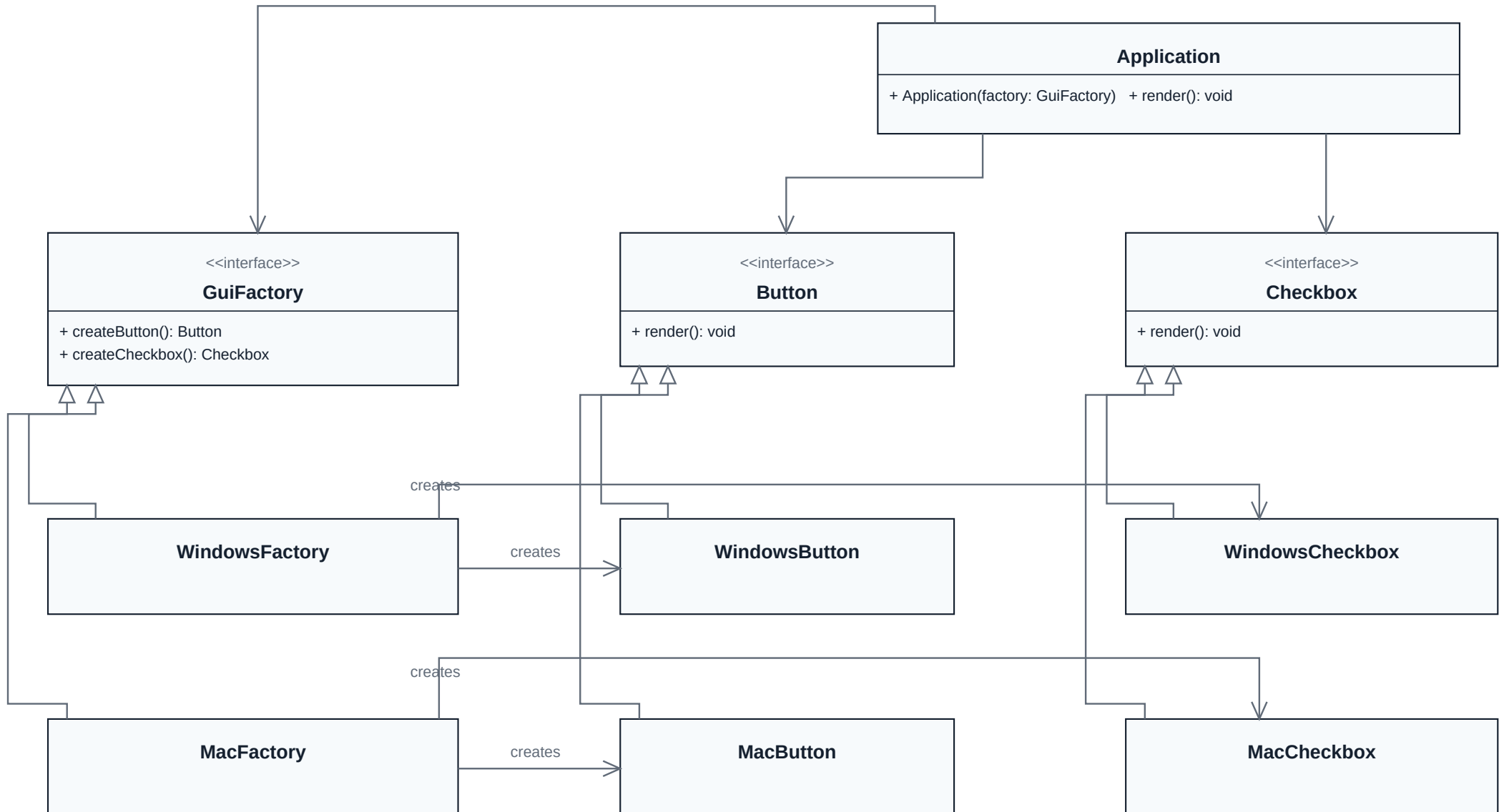
Example: `private final Button button;`

Factory Method UML



Hollow triangle: inheritance; dashed triangle: interface realization.
Dashed open arrow: dependency.

Abstract Factory UML



Dashed hollow triangle: implements. Dashed open arrow: dependency. Solid open arrow: product reference.